

SIMATS ENGINEERING ASSESSMENT TOOL 1

**COURSE NAME: ARTIFICIAL INTELLIGENCE
COURSE CODE: CSA1707**

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.

(BTL 5)

Assessment Tool Weightage: 50%

Annexure A

Design Task

Code of Conduct:

I ____ V.Lasya(192524186)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: v.lasya

1. HYBRID AI RECOMMENDATION ENGINE DESIGN

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

The proposed Hybrid AI Recommendation Engine combines content-based filtering and collaborative filtering in a distributed PySpark architecture. The system accepts large volumes of user-item interactions such as ratings, clicks, searches, purchases, viewing history and item metadata. Content-based filtering recommends items similar to those previously preferred by a user, while collaborative filtering identifies relationships between users and items from collective interaction patterns.

PySpark RDDs distribute the interaction dataset across multiple worker nodes. The raw interaction data can be represented as an RDD of tuples such as (user_id, item_id, rating). Transformations such as map(), filter(), groupByKey(), reduceByKey(), join() and mapValues() can calculate user profiles, item statistics and similarity scores in parallel. Actions such as count(), collect() and saveAsTextFile() execute the required computations. Partitioning the RDD by user or item allows related records to be processed efficiently and reduces unnecessary data movement.

The hybrid recommendation score can combine both approaches as $\text{FinalScore} = \alpha(\text{ContentScore}) + \beta(\text{CollaborativeScore})$, where α and β control their contribution. An intelligent recommendation agent continuously observes user behaviour, receives feedback such as clicks, ratings and skips, updates the user preference state, and adjusts recommendation weights. A monitoring agent can detect behaviour changes while a ranking agent selects the final recommendations.

The architecture should include a data-ingestion layer, distributed-processing layer,

recommendation layer, intelligent-agent layer and feedback loop. Caching frequently reused RDDs can reduce repeated computation. Fault tolerance is supported because RDDs can recompute lost partitions from their lineage. This design therefore provides scalable recommendation generation for a large number of users and items while allowing real-time adaptation through intelligent agents.

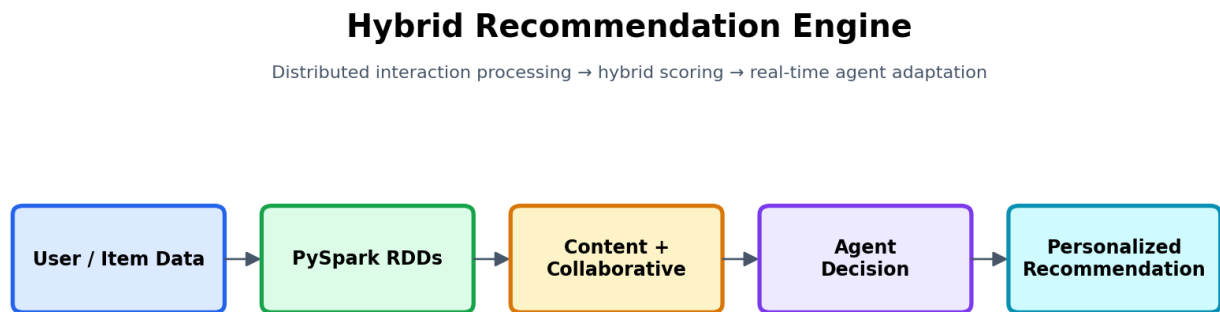


Figure 1: 1. HYBRID AI RECOMMENDATION ENGINE DESIGN system architecture and processing flow

2. SMART DISASTER DETECTION SYSTEM

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters such as floods or earthquakes from multi-source data streams including satellite images, sensors and social media. Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

The Smart Disaster Detection System is a distributed AI platform that combines satellite observations, physical sensors and social-media information. The objective is to identify abnormal environmental conditions quickly and generate an early warning before the disaster causes large-scale damage.

The ingestion layer receives sensor measurements such as water level, rainfall, temperature,

vibration and seismic activity. Satellite data can provide information about surface changes, flooded regions or damaged infrastructure. Social-media streams can provide location-tagged reports, emergency messages and public observations. Each source is converted into a common event representation containing timestamp, location, source and measured or extracted features.

PySpark RDDs distribute these records across worker nodes. `map()` can standardize records, `filter()` can remove invalid observations, `groupByKey()` can group events by geographical region, `reduceByKey()` can aggregate measurements, and `join()` can combine information from different sources. Data fusion combines evidence from multiple sources. For example, a sudden increase in water-level sensors together with satellite evidence of surface flooding and multiple local reports can produce a high-confidence flood event.

The intelligent-agent layer can contain a Sensor Monitoring Agent, Satellite Analysis Agent, Social Media Analysis Agent, Fusion Agent, Warning Agent and Response Coordination Agent. The Fusion Agent combines evidence and estimates confidence. The Warning Agent determines severity and sends alerts when conditions are satisfied. The Response Coordination Agent can recommend evacuation routes, resource allocation and priority zones.

The system should use spatial and temporal windows so geographically close events occurring within a relevant time interval are analysed together. RDD partitioning by region can improve parallel processing. Checkpointing and fault-tolerant processing increase reliability. The complete pipeline is multi-source ingestion → distributed preprocessing → feature extraction → data fusion → disaster classification → intelligent-agent coordination → early warning and response.

Smart Disaster Detection

Multi-source streams → distributed analysis → coordinated emergency response



Figure 2: 2. SMART DISASTER DETECTION SYSTEM system architecture and processing flow

3. AI-BASED FRAUD DETECTION FRAMEWORK

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

The AI-Based Fraud Detection Framework processes large-scale financial transactions using PySpark RDDs and distributed anomaly detection. Each transaction may contain transaction ID, customer ID, amount, timestamp, merchant, location, device information and transaction type. The system detects unusual patterns such as sudden high-value transactions, impossible travel, repeated failed transactions, unusual device usage and abnormal transaction frequency.

The transaction stream is converted into an RDD and partitioned across worker nodes. `map()` extracts features, `filter()` removes invalid transactions, `groupByKey()` forms customer histories, `reduceByKey()` calculates aggregate statistics, and `join()` combines transaction information with customer profiles or known fraud patterns. RDD caching can be used for frequently accessed customer histories or reference data.

Anomaly detection is distributed by calculating customer-level and transaction-level features in parallel. The system can calculate average transaction amount, transaction frequency, geographical distance between consecutive transactions and deviation from normal behaviour. An anomaly score is then computed for every transaction. Transactions above a threshold are sent to the intelligent-agent layer.

The agent architecture can include a Transaction Monitoring Agent, Customer Behaviour Agent, Pattern Detection Agent, Risk Scoring Agent and Alert Agent. The monitoring agent observes incoming transactions. The behaviour agent maintains a profile of normal activity. The pattern agent searches for suspicious relationships. The risk agent combines evidence into a fraud-risk score. The alert agent generates alerts for high-risk transactions and routes them for investigation.

A feedback loop is essential. Confirmed fraud cases can be added to the training dataset, while legitimate transactions incorrectly flagged as fraud can be used as negative feedback. This allows detection models and agent policies to improve over time. RDD distributed computation and fault-tolerant lineage allow the framework to process very large transaction datasets while maintaining scalable fraud analysis.

AI Fraud Detection Framework

Large-scale transactions → parallel anomaly scoring → collaborative fraud investigation

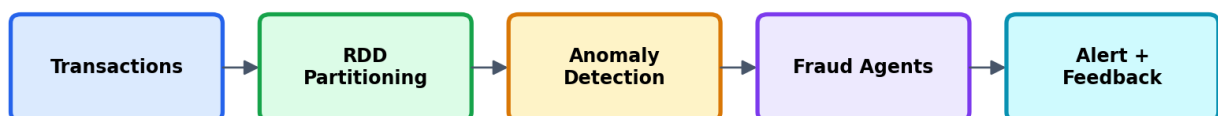


Figure 3: 3. AI-BASED FRAUD DETECTION FRAMEWORK system architecture and processing flow

4. AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM

Construct a distributed AI system using PySpark for optimizing supply chain operations including inventory, logistics and demand forecasting. Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

The Autonomous Supply Chain Optimization System integrates inventory records, customer orders, supplier information, transportation data, warehouse information and historical demand. The goal is to forecast demand accurately, maintain suitable inventory levels, optimize logistics and reduce total operational cost.

PySpark RDDs provide a distributed foundation for processing large historical and real-time datasets. An order RDD can be represented as (product_id, quantity, timestamp, location). map() transforms raw orders into useful features, filter() removes invalid records, groupByKey() groups demand by product or region, reduceByKey() calculates total demand, and join() combines demand information with inventory and supplier records. RDD partitioning by product or geographical region supports parallel processing.

The architecture can use separate intelligent agents for different decisions. A Demand Forecasting Agent estimates future demand from historical sales and seasonal patterns. An Inventory Agent decides reorder quantities and safety-stock requirements. A Logistics Agent selects transportation routes and shipment schedules. A Supplier Agent evaluates supplier availability, lead time and reliability. A Coordination Agent resolves conflicts between specialised agents and selects a globally efficient plan.

A typical process is collect demand → forecast future requirements → check inventory →

identify shortages or excess stock → evaluate supplier and transport options → select actions → monitor outcomes → update agent policies. The objective can consider purchase cost, transportation cost, inventory holding cost, shortage penalties and delivery time.

Distributed computing improves scalability because product, region and time partitions can be processed simultaneously. RDD lineage provides fault recovery if a worker fails. Caching can speed up iterative forecasting or optimisation operations. Intelligent agents make decentralized decisions, but coordination is necessary so local decisions do not create global inefficiencies. The system can reduce cost, improve delivery reliability and respond rapidly to changes in demand or supply.

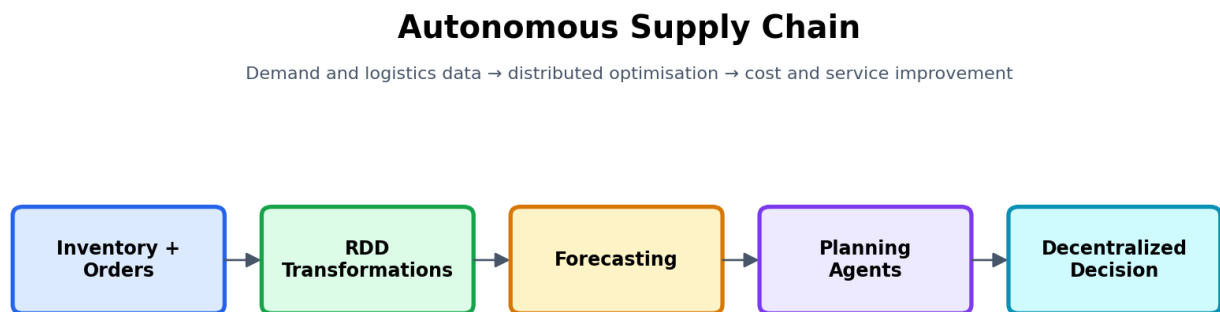


Figure 4: 4. AUTONOMOUS SUPPLY CHAIN OPTIMIZATION SYSTEM system architecture and processing flow

5. PERSONALIZED LEARNING ANALYTICS PLATFORM

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

Answer: The Personalized Learning Analytics Platform analyses student learning behaviour at scale and provides individualized educational content. The system collects data such as quiz scores, assignment marks, video watch time, page visits, question attempts, response time, topic mastery and learning activity history.

PySpark RDDs process the large volume of learner-event data generated by thousands of students. Each event can be represented as (student_id, course_id, topic_id, activity_type, score, timestamp). map() extracts useful features, filter() removes invalid events, groupByKey() generates learning histories, reduceByKey() calculates topic-level statistics, and join() combines student activity with course and content metadata. RDD caching improves repeated analytics operations.

The intelligent-agent architecture can include a Student Profiling Agent, Performance Analysis Agent, Content Recommendation Agent, Feedback Agent and Instructor Support Agent. The Student Profiling Agent maintains the learner's current knowledge state. The Performance Agent identifies weak and strong topics. The Content Recommendation Agent selects suitable learning resources. The Feedback Agent provides immediate guidance after assessments. The Instructor Support Agent identifies students who may require intervention.

Adaptive learning is achieved through a continuous feedback loop. If a student repeatedly performs poorly in a topic, the system can recommend simpler explanations, examples and additional practice. If a student demonstrates mastery, the system can recommend advanced material. Real-time events update the learner profile so recommendations do not depend only on old semester-level data.

For scalability, student records can be partitioned across worker nodes, allowing thousands of learners to be analysed in parallel. Fault-tolerant RDD lineage supports recovery from node failures. The platform can generate aggregate dashboards for instructors while preserving individualized recommendations for students. The complete pipeline is learning-event ingestion → distributed RDD processing → learner profiling → performance prediction → intelligent-agent decision → personalized content → feedback → continuous profile update.

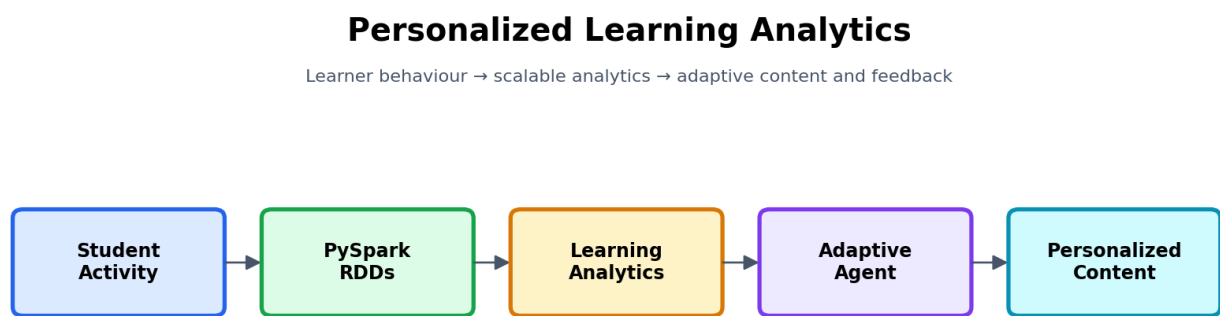


Figure 5: 5. PERSONALIZED LEARNING ANALYTICS PLATFORM system architecture and proc

RUBRICS FOR CALCULATION

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed

Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well- organized report	Mostly clear	Somewhat organized	Poor presentation